

Algorithmic Pricing in a Duopoly Gym

Vedant Mainkar

Supervised by:

Dr. Janusz Meylahn

Ben Meylahn

September 2024

Abstract

This report presents a customizable pricing gym environment designed to study algorithmic pricing and potential collusive behavior in duopoly markets. The gym features a flexible architecture with three main components: environment, agent, and metrics, enabling systematic investigation of how different pricing algorithms interact in various market settings. Key innovations include the ability to simulate interactions between discrete and continuous space algorithms and novel metrics for detecting collusive behavior. The framework provides valuable insights for both academic research and regulatory practices in algorithmic pricing.

Contents

1	Introduction	3
1.1	Anti-competitive behavior	3
1.2	Framework	4
1.2.1	Pricing Environment	4
1.2.2	Pricing Algorithm	5
2	Methods	5
2.1	Gym Environment Design	5
2.1.1	Environment Structure	6
2.1.2	Agent Architecture	6
2.1.3	Metrics	8
2.2	Environment Implemented	9
2.2.1	Logit demand model	9
2.3	Agents Implemented	10
2.3.1	QBase Class	11
2.3.2	Q-Learning	13
2.3.3	Decentralized Q-Learning	14
2.3.4	EXP3 Algorithm	16
2.3.5	Stochastic Gradient Descent	17
2.3.6	Proximal Policy Optimization	19
2.4	Metrics Implemented	22
2.4.1	Average prices and profits.	22
2.4.2	State Heat Map	23
2.4.3	Make adjacency	24
2.4.4	Reward-Punishment Scheme	25
2.4.5	Regret metric	26
3	Results	27
3.1	Experimental Setup	28
3.2	Experiment 1: Q-learning vs SGD	28
3.3	Experiment 2: Test for properties of collusion	29
4	Conclusion	31
A	Running Your Own Experiments	35
A.1	Environment Selection	35
A.2	Agent Selection	35
A.3	Metrics and Analysis	35
A.4	Example Configuration	36

1 Introduction

Anti-competitive behavior in markets poses threats to consumer welfare, market efficiency, and overall economic health. This behavior can lead to higher prices, reduced choices, and stifled innovation, ultimately disadvantaging consumers and hindering fair competition between firms [1, 2]. In recent years, the proliferation of algorithmic pricing strategies has intensified concerns regarding the potential for automated systems to engage in anti-competitive practices, either through explicit coordination or implicit parallelism [3].

1.1 Anti-competitive behavior

As more companies adopt algorithms to set and adjust the prices of their products, it becomes imperative to develop methodologies to detect anti-competitive behavior within these automated systems [4]. The use of algorithms in pricing introduces new dynamics and complexities that traditional regulatory frameworks may not fully address. Traditional regulatory frameworks were designed to monitor human decision-making and explicit communication between competitors, but algorithmic pricing systems can coordinate prices at inhuman speeds through complex ‘black box’ mechanisms without direct communication, making it difficult to detect and prove anti-competitive behavior under existing laws. Consequently, understanding whether certain pricing algorithms are capable of collusion and exhibit anti-competitive behavior is crucial for maintaining healthy market competition and protecting consumer interests.

Reinforcement Learning (RL), a branch of machine learning, offers powerful tools for developing and analyzing decision-making agents that learn optimal strategies through interactions with their environment. Since many firms use proprietary pricing algorithms which researchers don’t have access to, most of the research surrounding algorithmic collusion involves researchers building their own (usually RL) algorithms to test for collusive behavior (see for example ‘Collude and Compete’ [5], ‘Q-learning’ [3], ‘Decentralized Q-learning’ [6]).

Recent research has actively explored various aspects of algorithmic collusion. Abada and Lambin [7] investigated collusion in buying and selling storable goods through simple machine-learning algorithms, showing collusive behavior can emerge through imperfect exploration. Klein [8] demonstrated how reinforcement learning algorithms can converge to collusive equilibria in sequential competition with discrete prices. Abada et al. [9] focused on developing criteria for identifying potentially problematic learning algorithms that consistently maintain high prices. These studies highlight the growing concern about algorithmic collusion across different market contexts and regulatory frameworks.

This report introduces a customizable RL gym environment. The gym was inspired by the foundational work of den Boer and Meylahn [10], which provides a mathematical definition of algorithmic collusion. Their definition emphasizes the necessity of having a formal and precise understanding of what constitutes algorithmic collusion, particularly in the context of evolving antitrust laws.

The primary objective of the pricing gym is to test and evaluate whether

specific pricing algorithms demonstrate anti-competitive behavior in simulated market settings using mathematically motivated measures. This gym can simulate different kinds of algorithms running against each other with different state and action spaces. It is designed with three customizable components:

- **Environment:** Initializes the reward function and environment-dependent features such as Nash and Monopoly prices. This implementation uses the Logit demand model.
- **Agent:** Implements the specific learning algorithms. A few algorithms implemented are Q-learning, Stochastic gradient descent, and EXP3.
- **Metric:** Calculates and outputs information regarding the agents' performance and learning dynamics. A few metrics implemented are Average profits per agent, and the agents' external regret.

This design ensures flexibility, allowing users to incorporate their own environments, agents, and metrics while maintaining compatibility with pre-installed features. By facilitating the comparison of different algorithms across various environments, the gym serves as a valuable tool for researchers and practitioners to identify and understand the conditions under which algorithmic collusion may arise.

The structure of this report is organized as follows. Section 2.1 details the design of the gym environment, including the structure of the Environment, Agent, and Metrics components. Section 2.2 describes the specific environment implemented, with a focus on the Logit Demand (LD) model. Section 2.3 outlines the various algorithms implemented within the framework, such as Q-Learning, Decentralized Q-Learning, EXP3, and Stochastic Gradient Descent (SGD) agents. Section 2.4 explains the metrics used to evaluate agent performance and detect collusive behavior. Section 3 presents experiments conducted to analyze the behavior of different pricing algorithms and their propensity to collude. Section 4 provides concluding remarks and discusses potential directions for future research.

By systematically exploring the interactions between different learning algorithms and market environments, this report aims to contribute to the ongoing discourse on algorithmic collusion and offer insights that can inform both academic research and regulatory practices.

1.2 Framework

This section provides a broad framework and notation for the mathematical concepts used in the report.

1.2.1 Pricing Environment

A pricing environment is defined as a tuple $(N, \{A_i\}_{i \in N}, \{r_i\}_{i \in N}, d, S, P)$ where $N = \{1, \dots, n\}$ represents the set of agents/firms. For each agent $i \in N$, A_i

denotes their action space representing possible prices they can charge, with $r_i : A_1 \times A_2 \times \dots \times A_n \rightarrow \mathbb{R}$ being their reward/profit function. The demand function $d : \mathbb{R}^n \rightarrow \mathbb{R}^n$ maps price vectors $p = (p_1, \dots, p_n)$ to quantities demanded. The dynamics of the environment are characterized by the state space (which is agent specific, but the most common one used is $S = A_1 \times A_2 \times \dots \times A_n$ representing joint price configurations), and reward functions $r_i : S \times A_1 \times \dots \times A_n \rightarrow \mathbb{R}$ that give profits for each agent based on state-action pairs. The reward distribution $P : S \times A_1 \times \dots \times A_n \rightarrow \mathcal{P}(\mathbb{R})$ maps state-action pairs to a probability distribution over rewards, where P can be either deterministic or stochastic. In this gym, the pricing environment is a duopoly, meaning only two algorithms play against each other.

1.2.2 Pricing Algorithm

An algorithm in a pricing environment is defined as a tuple (P, H, μ, π) where P represents the set of all possible policies. Each policy maps the agent's information set to a distribution over actions, formally $\pi : H \rightarrow \mathcal{P}(A)$, where H is the history space containing all observable past states, actions, and rewards $h_t = (s_0, a_0, r_0, \dots, s_t)$. The learning mechanism $\mu : H \times A \times \mathbb{R} \rightarrow P$ updates the policy based on observed history, actions, and rewards. The current policy $\pi \in P$ can be either deterministic ($\pi : H \rightarrow A$) or stochastic ($\pi : H \rightarrow \mathcal{P}(A)$), and may incorporate various learning parameters such as exploration rates, learning rates, or value function approximators. The algorithm's objective is typically to maximize the expected cumulative discounted reward $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$, where $\gamma \in [0, 1]$ is a discount factor determining the trade-off between immediate and future rewards. Currently, the gym only uses a one step history of the two pricing agents to update its policy.

2 Methods

2.1 Gym Environment Design

The gym consists of three customizable components, the Environment - where the reward function and the environment dependent features, such as the Nash and Monopoly prices, are initialized. The Agent - where the specific learning algorithm is implemented. The Metric - where information about the specified agents and their learning dynamics in the environment can be calculated and output.

The main idea of this project is to test different algorithms against each other, since in a situation where no communication between firms is present, self-play (both firms using the same learning algorithms) is unlikely. This way, we can test whether properties of collusion studied with algorithms playing against their copies, also hold against other algorithms. Therefore, the code is written in such a way that a user can add their own environment, agent, or metric in a specific format and have it work with the preinstalled features.

2.1.1 Environment Structure

The environment structure is a key component of the gym, the environment is responsible for calculating the profits based on the prices chosen by the two agents. An environment needs to be initialized with two parameters: **tmax** (int) and **tstable** (int), where **tmax** is the maximum number of steps a single learning trajectory runs for, and **tstable** is the number of steps the for which the algorithms are stable in order to determine whether sufficient learning has occurred. This stability value is expanded upon in the Agent architecture. An environment needs to have two core functions in order to maintain compatibility with the existing framework.

- **compute_profits() Function**

The `compute_profits()` function calculates the profits for each firm based on their chosen prices.

Inputs: p (`numpy.array`): A numpy array of prices set by the 2 firms, where p_i is the price of firm $i \in \{1, 2\}$.

Outputs: π_i (`numpy.array`): A numpy array of profits for each firm, where π_i is the profit of firm $i \in \{1, 2\}$.

- **compute_p_competitive_monopoly() Function**

The `compute_p_competitive_monopoly()` function computes both competitive and monopoly prices for the firms. The method for calculating these prices is written in section 2.2. These values are then used in a metric explained in section 2.4 to calculate degree of collusion.

Inputs: None. This function utilizes the class attributes to perform its calculations.

Outputs: $p_{competitive}$, $p_{monopoly}$:

- $p_{competitive}$ (`numpy.array`): An array of competitive prices for each firm.
- $p_{monopoly}$ (`numpy.array`): An array of monopoly prices for each firm.

2.1.2 Agent Architecture

The agent architecture within this framework is structured around three fundamental functions that enable decision-making, state resetting, and strategy updating. These functions ensure that agents can effectively interact within the game environment, learn from experiences, and adapt their behaviors over time. Each agent is required to implement the following core functions:

- **pick_strategies() Function**

The `pick_strategies()` function is responsible for selecting the agent's action based on the current state of the game.

Inputs:

- **game** (`GameEnvironment`): The game environment object. This is only used when using Q-value initializations based on average reward.
- **previous_prices** (`numpy.array`): A NumPy array of shape (2,) representing the previous price pair chosen by the agents, a one step history of the state. During the first step, the starting action is the first index for an algorithm with a discrete action space, or the median value for a continuous action space.
- **t** (`int`): The current time step.

Outputs: **action** (`numpy.array`): A NumPy array of shape (1,) representing the chosen action as prices, determined by the agent’s learned policy and exploration strategy.

This function enables the agent to make informed decisions based on its accumulated knowledge and exploration mechanisms, guiding its actions within the game.

- **reset() Function**

The **reset()** function reinitializes the agent’s internal states, preparing it for a new learning trajectory. For every agent the reset function must reset each value changed during the learning process to its initial value (possibly random). For example, the learned Q-learning values are replaced by a new independent initialization.

Inputs: **game** (`GameEnvironment`): The game environment object.

Outputs: None

This function is essential for running multiple learning trajectories, ensuring that each new trajectory begins with a clean state free from previous experiences.

- **update_function() Function**

The **update_function()** function updates the agent’s strategy based on the latest observed outcomes.

Inputs:

- **game** (`GameEnvironment`): The game environment object. This is only used when using Q-value initializations based on average reward.
- **previous_prices** (`numpy.array`): A NumPy array of shape (2,) representing the previous price pair chosen by the agents.
- **pi** (`numpy.array`): A NumPy array of shape (2,) representing the observed payoffs.
- **stable** (`int`): The stability counter indicating the number of consecutive stable updates.
- **t** (`int`): The current time step.
- **tol** (`float`): The convergence tolerance.

Outputs:

- `updated_strategy` (`numpy.array`): A NumPy array representing the updated strategy.
- `stability_counter` (`int`): The updated stability counter.

The `stability_counter` keeps track of certain agent dependent checks in order to determine when the agent has stabilized. For example, the `stability_counter` for Q-learning increases by one if all values in the Q-matrix have not changed beyond a specified tolerance. If they have, then the `stability_counter` resets to zero. If the counter reaches a specified value in the environment, then the agent is considered stabilized.

This function allows agents to refine their strategies by incorporating new information from observed payoffs. For example, for Q-learning it updates the Q-values based on the Q-value update function shown in section 2.3.2. Additionally, it updates the stability counter, which is utilized by the `check_end()` function within the environment class to determine convergence.

2.1.3 Metrics

The **Metrics** component serves as the backbone for running and managing simulations within the framework. It encompasses the simulation base class, which orchestrates the execution of game iterations between agents, monitors stability criteria, and aggregates results for analysis. Using the results obtained from running these simulations, many functions can be called to analyze the behavior of the agents over all the trajectories.

Simulation Base Class

The **Simulations** class encapsulates the functionality required to perform and manage multiple simulation runs. It handles the initialization of simulations, execution of iterative game processes, and aggregation of results for subsequent analysis. The user must first initialize the class with 6 inputs: the environment, the two agents, the number of trajectories to run, whether to save and record the Agent states (All the information stored the agent class at a specific step during learning), and the time intervals for all recorded data. The last two inputs are required for memory saving reasons. Saving Agent states is very memory heavy and is only used in one of the metric functions. The time interval defines how many time steps will be skipped between each action/Agent state recorded. Therefore if the time interval is 100, the class only records every hundredth action by the agents per trajectory.

get_values() Method

Runs simulations (if not already executed) and retrieves actions taken by both agents, the Q-values (if applicable), and the state of each agent after each simulation.

Inputs:

- None

Outputs:

- `a1_list` (`list of list`): Actions taken by Agent1 across all simulations.
- `a2_list` (`list of list`): Actions taken by Agent2 across all simulations.
- `Agent1_list` (`list`): List of Agent1 instances after each simulation.
- `Agent2_list` (`list`): List of Agent2 instances after each simulation.

This method processes the results from all simulations, extracting the sequence of actions for each agent and storing the final states of the agents post-simulation. This data is crucial for subsequent analysis and evaluation of agent performance.

This systematic approach ensures that simulations are conducted uniformly, enabling analysis of agent behaviors and strategic dynamics within the game environment.

2.2 Environment Implemented

2.2.1 Logit demand model

The LD (Logit demand) model is designed to simulate and analyze competitive interactions within an oligopolistic market where multiple firms engage in strategic pricing of differentiated products.

Mathematical Framework

The LD model is grounded in a mathematical framework that defines the relationships between pricing strategies, demand, and profits within an oligopolistic market. The core components of the model are as follows:

Demand Function

The demand faced by each firm is modeled using a logit-based discrete choice framework, which captures the probabilistic nature of consumer preferences among differentiated products and an outside option. The demand for firm i is given by:

$$d_i(p) = \frac{e^{(a-p_i)/\mu}}{\sum_{j=1}^n e^{(a-p_j)/\mu} + e^{a_0/\mu}}, \quad i \in \{0, 1\}.$$

In this equation, $p = (p_1, p_2, \dots, p_n)$ represents the vector of prices set by the n competing firms. The parameter a denotes the base utility of the products, reflecting their inherent attractiveness to consumers. The variable μ indicates the degree of product differentiation, which measures how substitutable the products are from one another. Additionally, a_0 is the utility of the outside option, representing consumers' alternative choice to purchasing any of the available products. In this implementation, the demand is symmetric, however, it is certainly possible to have asymmetric demand functions for both agents by modifying the `compute_profits()` function.

This demand function captures the inverse relationship between price and quantity demanded, moderated by the parameter μ , which represents the sensitivity of consumers to price changes. A lower μ implies higher sensitivity, leading to more elastic demand, while a higher μ indicates less sensitivity and more inelastic demand.

Profit Function

Each firm's profit is determined by the price it sets, the resulting quantity demanded, and its marginal cost of production. The profit for firm i is defined as:

$$\pi_i(p) = (p_i - c) \cdot d_i(p), \quad i \in \{0, 1\},$$

where c is the constant marginal cost of production for each firm and $d_i(p)$ is the demand for firm i 's product as derived from the demand function.

This linear profit function assumes that the profit margin per unit sold is constant and directly proportional to the quantity demanded. It encapsulates the economic principle that profit increases with higher pricing, provided that the quantity demanded does not decrease excessively due to higher prices.

First-Order Conditions (FOC)

To identify the equilibrium prices where each firm maximizes its profit given the pricing strategies of its competitors, the model employs first-order conditions derived from profit maximization. For a competitive equilibrium, the FOC for each firm i is given by:

$$\frac{\partial \pi_i}{\partial p_i} = 0 \implies 1 - \frac{(p_i - c)(1 - d_i(p))}{\mu} = 0, \quad i \in \{0, 1\}.$$

For a monopolistic scenario, where a single firm controls all prices in the market, the FOC adjusts to account for the impact of the firm's pricing on the demand for its products:

$$\frac{\partial}{\partial p_i} \sum_{j=1}^n \pi_j = 0 \implies \sum_{j=1}^n \left(1 - \frac{(p_i - c)(1 - d_i(p))}{\mu} + \frac{(p_j - c)d_j(p)}{\mu} \right) = 0, \quad \forall j \neq i.$$

These conditions are solved numerically using methods such as the Newton-Raphson technique (e.g., via `fsolve`) to determine the equilibrium prices.

2.3 Agents Implemented

Five different agents were implemented in the gym. All agents were implemented in a standard form such that any agent can play against another. Currently the gym only supports two agents with a one step history of prices. All the agents have their state and action spaces in their individual classes. Therefore, during the learning, neither the opposing agent, nor the environment see the agents state and action spaces. All they see is the price chosen by the opposing agent. This design choice was made so that the gym can more faithfully simulate the

market where firms might not know the opponent's decision making strategies or their prices.

2.3.1 QBase Class

Mathematical Framework

The **QBase** class serves as the foundational framework for Q-Learning agents operating within a pricing game environment. It encapsulates the essential components required to initialize and manage the action and state spaces, profit matrices, Q-value estimations, and visitation counts. The mathematical principles underpinning the **QBase** class are delineated below:

Action Space Construction

The action space represents the set of possible pricing strategies available to each agent. Mathematically, the action space for agent i is defined as:

$$A_i = \{p_{i1}, p_{i2}, \dots, p_{ik}\},$$

where p_{ij} denotes the j -th discrete price point for agent i , and k is the total number of discrete price points.

State Space Initialization

The state space encompasses all possible configurations of actions taken by the agents. For a two-agent game, the state space S is defined as the Cartesian product of the individual action spaces:

$$S = A_1 \times A_2$$

Each state $s \in S$ is represented by a tuple (p_1, p_2) , where $p_1 \in A_1$ and $p_2 \in A_2$.

Q-Value Initialization

The Q-values $Q(s, a)$ estimate the expected cumulative rewards for each state-action pair. Initialization of Q can follow different strategies:

Uniform Initialization

$$Q(s, a) \sim \mathcal{U}(0, 1),$$

where each $Q(s, a)$ is sampled uniformly from the interval $[0, 1]$.

Zero Initialization

All values are set to 0.

$$Q(s, a) = 0 \quad \forall s \in S, \forall a \in A.$$

Mean Payoff Initialization

For each state-action pair (s, a) , $Q(s, a)$ is initialized based on the average payoff across the opponent's actions:

$$Q(s, a) = \frac{1}{|A_{-i}|} \sum_{a' \in A_{-i}} \pi_i(a, a') \cdot \frac{1}{1 - \gamma},$$

where A_{-i} denotes the action space of the opponent, and γ is the discount factor.

Using this initialization requires the agent to know the reward function of the environment. It is not very realistic to assume that firms know exactly how the market will respond to the prices, but they can estimate this through historical data.

State Indexing

Since the agents are designed to play against opponents using unknown action spaces, the opponents may set a price which was not in the Q-learner's state space. Therefore, this state indexing algorithm is used to determine how the Q-learner should read and update its values based on an unknown price by the opponent. Efficient lookup and updating of Q-values require mapping continuous price actions to discrete indices. This is achieved through the following functions:

1. Index Retrieval

For a given price p in agent i 's action space A_i , the corresponding index ℓ is:

$$\ell = \arg \min_j |p - p_{ij}|.$$

2. State Augmentation

When a new price p' is introduced that is not within a predefined tolerance τ of existing prices, the state space is augmented to include p' :

$$\text{If } \forall p \in S, |p - p'| > \tau \implies S' = S \cup \{p'\},$$

Correspondingly, the Q-value matrix Q and visitation count matrix num are expanded to accommodate the new state. The new set of rows appended are also initialized using the same initialization type chosen for the original Q-matrix.

Exploration Tolerance (τ)

To manage the granularity of the state space, a tolerance level τ is defined to determine whether a new price should be considered distinct or lumped with existing states:

$$\tau = \text{lump_tol}.$$

A new price p' is considered similar to an existing price p if $|p' - p| \leq \tau$, thereby preventing an unbounded increase in the state space.

Dimensionality of Q-Values

For a two-agent game, the dimensionality of the Q-value matrix Q is:

$$\text{Shape}(Q) = (|A_1|, |A_2|, |A_1|),$$

where $|A_i|$ denotes the number of discrete actions available to agent i . This structure allows each agent to reference the Q-values corresponding to their own actions and the actions of their opponents.

Action Space Augmentation

When the state space is augmented to include a new price, the Q-value matrices Q and Q_{val} are expanded accordingly. This augmentation ensures that the learning agent can accommodate new strategies without disrupting the existing Q-value estimations:

$$Q' = \text{Augment}(Q), \text{ and } Q'_{\text{val}} = \text{Augment}(Q_{\text{val}})$$

where $\text{Augment}(\cdot)$ denotes the process of appending new action dimensions to the Q-value matrices using the predefined initialization (uniform, zero, mean).

Visitation Count Matrix (num)

The visitation count matrix tracks the number of times each state has been encountered during the learning process, i.e.,

$$\text{num}(s) = \text{Number of visits to state } s.$$

This matrix is initialized to zero and incremented as states are visited.

2.3.2 Q-Learning

Mathematical Framework

The **Q_Learning** class extends the foundational **QBase** class to implement Q-Learning. This algorithm is studied in various papers about pricing and collusion in oligopolies [3, 11]. The core components of the Q-learning framework in this context are outlined below:

Q-Function

The Q-function, denoted as $Q(s, a)$, quantifies the expected cumulative reward an agent can achieve by taking action a in state s and subsequently following an optimal policy. Mathematically, it is defined as:

$$Q(s, a) = \mathbb{E}_{\pi^*} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right], \quad (1)$$

where $s \in S$ represents the current prices set by each agent, $a \in A$ is the price chosen by the agent, $\gamma \in (0, 1)$ is the discount factor determining the importance of future profit, r_t is the profit received at time step t , and π^* is the optimal policy of the agent.

The state s encapsulates the relevant information about the game's current configuration, including actions taken by all players and any other pertinent variables. In this implementation, the state s is the one step history of the prices charged by both agents.

Q-Learning Update Rule

Q-learning updates the Q-function iteratively using the Bellman equation, enabling the agent to learn optimal action-values through experience. The update rule for the Q-value corresponding to a state-action pair (s, a) is given by:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where $\alpha \in (0, 1)$ is the learning rate dictating the extent to which new information overrides existing estimates, r is the immediate reward received after taking action $a \in A$ in state $s \in S$, $s' \in S$ is the subsequent state resulting from action a , and $\max_{a'} Q(s', a')$ represents the maximum estimated Q-value for the next state $s' \in S$ over all possible actions $a' \in A$.

This recursive update ensures that the Q-function converges to the optimal action-value function $Q^*(s, a)$, provided that the learning rate α decays appropriately and sufficient exploration of state-action space is achieved [12]. Note that this convergence is only provable in a single agent setting.

Epsilon-Greedy Strategy

To balance exploration and exploitation, the `Q_Learning` class employs an epsilon-greedy policy for action selection. The agent chooses actions according to the following rules:

$$\pi(a|s) = \begin{cases} \frac{\epsilon_t}{|A|} & \text{for all } a \in A \\ \frac{\epsilon_t}{|A|} + (1 - \epsilon_t) & \text{for } a = \arg \max_{a'} Q(s, a') \end{cases} \quad (2)$$

where ϵ_t is the exploration rate at time step t , typically decaying over time to reduce exploration as the agent becomes more confident in its learned Q-values.

Following Calvano [3], the exploration rate ϵ_t decays exponentially as:

$$\epsilon_t = e^{-\beta t},$$

with β being the decay rate parameter. This decay mechanism ensures that the agent explores extensively in the early stages of learning while gradually shifting towards exploitation of the best-known actions as learning progresses.

This is one choice for setting the exploration parameter, one can also use a fixed *epsilon* value or some other slower decaying function like a polynomial function.

Stabilization Criteria:

The Q-learning algorithm is considered to have stabilized when the Q-function changes insignificantly between successive updates, typically measured by:

$$\|Q_{\text{new}} - Q_{\text{old}}\|_{\infty} < \text{tol},$$

where `tol` is a predefined tolerance threshold. This ensures that the Q-function has stabilized, indicating that the agent has sufficiently learned the optimal action-values for the given environment or the exploration has stopped.

2.3.3 Decentralized Q-Learning

Mathematical Framework

The `Dec.Q` class extends the foundational `QBase` class to implement Decentralized Q-Learning [13]. This algorithm is designed to facilitate reinforcement learning for multiple agents operating independently in a strategic environment. The mathematical structure of the `Dec.Q` class integrates principles from

Q-Learning and SARSA (State-Action-Reward-State-Action), augmented with batch updates to enhance learning efficiency and stability. The core components of this mathematical framework are detailed below:

Q-Function for Decentralized Agents

In a decentralized setting, each agent maintains its own Q-function, denoted as $Q_i(s, a_i)$, where i indexes the agent, s represents the current state of the environment, and a_i is the action taken by agent i . The Q-function is the same as defined before in the Q-learning implementation (1).

Epsilon-Greedy Strategy

To balance exploration and exploitation, the agent employs an epsilon-greedy policy for action selection with a fixed exploration rate of ϵ . This algorithm also uses the same policy equation (2) in Section 2.3.1.

Batch SARSA Update Rule

The **Dec.Q** class integrates batch updates into the SARSA learning algorithm to enhance the stability and convergence of the Q-function estimates [14]. The SARSA update rule for a state-action pair (s, a_i) is given by:

$$Q_{val_i}(s, a_i) \leftarrow (1 - \alpha_i)Q_{val_i}(s, a_i) + \alpha_i(r_i + \gamma Q_{val_i}(s', a'_i)) \quad (3)$$

where $\alpha_i \in (0, 1)$ is the learning rate for agent i , r_i is the immediate reward received by agent i after taking action $a_i \in A$ in state $s \in A$, $s' \in S$ is the next state resulting from the joint actions of all agents, and $a'_i \in A$ is the action taken by agent i in the next state $s' \in S$.

In the context of batching, these updates are accumulated over a predefined number of steps (determined by the batch size B), and then applied collectively to the Q-function. These updates are therefore more stable due to less variance in the updates.

Adaption Phase and Batch Updates

The **Dec.Q** class incorporates an adaptation phase that synchronizes the primary Q-function Q with a secondary Q-function Q_{val} based on accumulated updates. During the batching phase, the agent plays the fixed policy based on the Q matrix and applies the Q-learning update to Q_{val} . At every B time step, the Q matrix updates its values to the Q_{val} matrix with a probability of $\gamma \in (0, 1)$. Therefore, at the end of every batch,

$$Q_{act_i}(s, a_i) = \begin{cases} Q_{val,i}(s, a_i) & \text{with probability } 1 - \lambda, \\ Q_{act_i}(s, a_i) & \text{with probability } \lambda. \end{cases}$$

This probabilistic update ensures that the Q-function integrates the latest learned values while retaining a degree of stability from previous estimates.

Visitation Count and Learning Rate Adjustment

The visitation count matrix $\text{num}(s, a_i)$ tracks the number of times each state-action pair (s, a_i) has been encountered during a batch. The learning rate α_i is dynamically adjusted based on this count:

$$\alpha_i = \frac{1}{\text{num}(s, a_i) + 1}$$

This adaptive learning rate is for each batch, and ensures that updates become smaller as the agent gains more experience with specific state-action pairs, promoting convergence by reducing the impact of new information on well-established estimates.

2.3.4 EXP3 Algorithm

Mathematical Framework

The `Exp3` class extends the foundational `QBase` class to implement the EXP3 (Exponential-weight algorithm for Exploration and Exploitation) algorithm [15]. This framework is designed to facilitate reinforcement learning for agents engaged in strategic interactions. Exp3 is a multi-armed bandit, which represents the simplest form of reinforcement learning, dealing with a single-state scenario where the agent learns to choose between different actions (arms) to maximize cumulative rewards over time. The core components of this mathematical framework are detailed below.

Weight Vector Initialization

In the EXP3 algorithm, each action is associated with a weight that determines its probability of being selected. The weight vector $\mathbf{w}$ is initialized uniformly to ensure unbiased exploration at the outset:

$$\mathbf{w} = [1, 1, \dots, 1]^\top \in \mathbb{R}^K,$$

where K is the number of available actions.

Probability Distribution over Actions

The probability $P(a)$ of selecting a particular action a is derived from the current weights and incorporates a mixing parameter γ that controls the exploration rate:

$$P(a) = (1 - \gamma) \frac{w(a)}{\sum_{a'=1}^K w(a')} + \frac{\epsilon}{K},$$

where $w(a)$ is the weight corresponding to action a , $\epsilon \in (0, 1)$ as the static exploration rate, and K is the total number of actions.

Action Selection Mechanism

At each time step t , an agent selects an action a_t based on the probability distribution $P(a)$:

$$a_t \sim P(a).$$

Reward Estimation

Upon selecting an action a_t and observing the corresponding reward r_t , the agent computes an estimated reward $\hat{r}_t$ to account for the probability of selecting that action:

$$\hat{r}_t = \frac{r_t}{P(a_t)}.$$

This unbiased estimator ensures that the algorithm correctly accounts for the exploration component in the probability distribution.

Weight Update Rule

The weights are updated multiplicatively based on the estimated rewards, incorporating the learning rate η :

$$w(a_t) \leftarrow w(a_t) \cdot \exp\left(\frac{\eta \cdot \hat{r}_t}{K}\right),$$

where η controls the influence of observed rewards on weight updates and K is the total number of actions providing normalization.

This update rule ensures that actions yielding higher estimated rewards receive exponentially increasing weights, thereby increasing their probability of being selected in future time steps.

Normalization of Weights

To prevent the weights from growing unbounded and to maintain numerical stability, the weight vector $\mathbf{w}$ is normalized every time step:

$$\mathbf{w} \leftarrow \frac{\mathbf{w}}{\|\mathbf{w}\|_1},$$

where $\|\mathbf{w}\|_1$ denotes the L1 norm of the weight vector, effectively ensuring that the sum of weights remains constant.

Stabilization Criteria

Stabilization in the EXP3 algorithm is determined by monitoring the stability of the weight vector $\mathbf{w}$ over successive iterations. Specifically, the algorithm is considered to have converged when the change in the weight vector between consecutive updates falls below a predefined tolerance threshold tol . Mathematically, stabilization is achieved when:

$$\|\mathbf{w}_{\text{new}} - \mathbf{w}_{\text{old}}\|_\infty < \text{tol},$$

where $\|\cdot\|_\infty$ denotes the L_∞ norm, and tol is the tolerance threshold. To ensure stability, this condition must hold for a `tstable` number of consecutive time steps, indicating that the weight vector has stabilized and the policy has sufficiently adapted to the environment. This stability implies that the agent has effectively balanced exploration and exploitation, having learned the optimal action probabilities within the given tolerance.

2.3.5 Stochastic Gradient Descent

Stochastic gradient descent (SGD) for pricing [16] was also implemented in the gym. This allows us to test algorithms with continuous state spaces against discrete state spaces, as SGD algorithms exist for both discrete and continuous action spaces.

Discrete Action Spaces

Mathematical Framework

For discrete action spaces, the agent maintains a preference vector $\theta(s) \in \mathbb{R}^k$ for each state s , where k is the number of discrete actions. The policy is defined using the softmax function:

$$\pi(a \mid s; \theta) = \frac{\exp\left(\frac{\theta_a(s)}{\tau}\right)}{\sum_{a'=1}^k \exp\left(\frac{\theta_{a'}(s)}{\tau}\right)},$$

where $\tau \in (0, \infty)$ is the temperature parameter that controls the randomness of action selection.

Action Selection

Given a state s , the agent selects an action a based on the softmax probabilities:

$$a \sim \pi(a \mid s; \theta).$$

This stochastic selection ensures a balance between exploration and exploitation, guided by the temperature parameter τ .

Policy Parameter Update via Stochastic Gradient Ascent:

The policy parameters $\theta_a(s)$ are updated using the gradient of the log-policy weighted by the observed reward:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a \mid s; \theta) r,$$

where $\alpha \in (0, 1)$ is the learning rate, r is the observed reward, and $a \in A$ is the action taken by the agent.

Stabilization Criteria

The SGD-based policy updates are considered to have converged when the change in policy parameters falls below a predefined tolerance:

$$\|\theta_{\text{new}} - \theta_{\text{old}}\|_2 < \text{tol},$$

where tol is the tolerance threshold. This ensures that the policy has stabilized, indicating that the agent has sufficiently learned the optimal action preferences for the given environment.

Continuous Action Spaces

Mathematical Framework

For continuous action spaces, the agent employs a Gaussian policy characterized by a mean $\mu(s)$ and a standard deviation $\sigma(s)$:

$$\pi(a \mid s; \theta) = \mathcal{N}(a \mid \mu(s), \sigma(s)^2).$$

Here, $\theta = (\mu(s), \sigma(s))$ are the policy parameters governing the distribution from which actions are sampled for each state.

Action Selection

In continuous action spaces, the agent samples an action a from the Gaussian distribution defined by the current policy parameters:

$$a \sim \mathcal{N}(\mu(s), \sigma(s)^2).$$

The sampled action may be clipped to lie within predefined bounds if necessary:

$$a = \text{clip}(a, a_{\text{low}}, a_{\text{high}}),$$

where a_{low} and a_{high} define the permissible action range.

Policy Parameter Update via Stochastic Gradient Ascent

The policy parameters $\mu(s)$ and $\sigma(s)$ are updated using the gradients of the log-probability with respect to these parameters:

$$\begin{aligned}\mu(s) &\leftarrow \mu(s) + \alpha \cdot r \cdot \left(\frac{a - \mu(s)}{\sigma(s)^2} \right), \\ \sigma(s) &\leftarrow \sigma(s) + \alpha \cdot r \cdot \left(\frac{(a - \mu(s))^2}{\sigma(s)^3} - \frac{1}{\sigma(s)} \right).\end{aligned}$$

To ensure numerical stability and prevent the standard deviation from becoming non-positive, the update rule includes a constraint:

$$\sigma(s) = \max(\sigma(s), \epsilon),$$

where ϵ is a small positive constant.

2.3.6 Proximal Policy Optimization

Mathematical Framework

The `PPO_Agent` class implements the Proximal Policy Optimization (PPO) algorithm, accommodating both discrete and continuous action spaces. PPO is a policy gradient method that aims to improve the stability and reliability of reinforcement learning by restricting the policy updates to be within a small trust region [17]. The mathematical foundation of the `PPO_Agent` class is detailed below.

Policy and Value Function Parameterization

The agent utilizes two neural networks:

- *Actor Network* ($\pi_\theta(a | s)$): Parameterized by θ , this network outputs either a probability distribution over actions in discrete action spaces or the parameters of a Gaussian distribution (mean and standard deviation) in continuous action spaces.

- *Critic Network* ($V_\phi(s)$): Parameterized by ϕ , this network estimates the state-value function, providing a baseline for advantage estimation. This can be represented as $\max_a Q(S, a)$ in terms of Q-values.

Policy Representation:

- *Discrete Action Spaces:*

The policy outputs a probability distribution over actions using a softmax function:

$$\pi_\theta(a | s) = \frac{\exp(f_\theta(s, a))}{\sum_{a' \in A} \exp(f_\theta(s, a'))},$$

where f_θ is the actor network that maps states to logits (unnormalized outputs of the neural network), $f_\theta(s)_a$ is the logit corresponding to action a given state s , and A is the set of possible actions.

- *Continuous Action Spaces:*

The policy is represented by a Gaussian distribution:

$$\pi_\theta(a | s) = \mathcal{N}(a | \mu_\theta(s), \sigma_\theta^2(s)),$$

where $\mu_\theta(s)$ and $\sigma_\theta(s)$ are the mean and standard deviation output by the actor network for state s .

Experience Buffer:

The agent collects experiences over multiple time steps and stores them in a buffer. Each experience consists of:

$$\{s_t, a_t, r_t, s_{t+1}, \log \pi_{\theta_{\text{old}}}(a_t | s_t)\},$$

where s_t is the state at time t , a_t is the action taken at time t , r_t is the reward received after taking action a_t , s_{t+1} is the next state, and $\log \pi_{\theta_{\text{old}}}(a_t | s_t)$ is the log probability of the action under the old policy parameters.

Advantage Estimation:

The advantage function Adv_t quantifies the relative value of an action compared to the expected value of the state. It is estimated using the critic network:

$$Adv_t = \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t),$$

or using Generalized Advantage Estimation (GAE) for improved bias-variance trade-off:

$$Adv_t = \sum_{l=0}^{\infty} (\gamma \sigma)^l \delta_{t+l},$$

where $\sigma \in (0, 1)$ is the GAE parameter.

Policy Update Objective:

PPO seeks to maximize a surrogate objective function that constrains the new policy from deviating too much from the old policy. The clipped surrogate objective $L^{\text{CLIP}}(\theta)$ is defined as:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) Adv_t, \text{clip}(r_t(\theta), 1 - \xi, 1 + \xi) Adv_t)],$$

where $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ is the probability ratio, ξ is the clip parameter that limits the extent of policy updates, and the clip function constrains $r_t(\theta)$ to the interval $[1 - \xi, 1 + \xi]$. This objective function ensures that the new policy does not diverge significantly from the old policy while still allowing sufficient exploration.

Value Function Update:

The critic network is updated by minimizing the mean squared error between the predicted value and the observed returns:

$$L^{\text{VF}}(\phi) = \mathbb{E}_t \left[(V_\phi(s_t) - R_t)^2 \right],$$

where R_t is the cumulative return from time t :

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

In practice, R_t is estimated using the collected rewards and the critic’s value estimates.

Total Loss Function:

The total loss $L(\theta, \phi)$ combines the policy surrogate loss and the value function loss, often including an entropy bonus S to encourage exploration:

$$L(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\phi) + c_2 S(\theta),$$

where c_1 and c_2 are coefficients balancing the value loss and entropy bonus. $S(\theta)$ is the entropy of the policy, defined differently for discrete and continuous actions. For discrete actions, the entropy is defined as:

$$S_\theta(s_t) = - \sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t).$$

For continuous actions with a Gaussian policy, the entropy is defined as:

$$S_\theta(s_t) = \frac{1}{2} \log(2\pi e \sigma_\theta^2(s_t)).$$

Action Selection:

During interaction with the environment, actions are selected based on the current policy.

For discrete action spaces, the agent samples an action a_t according to the probability distribution:

$$a_t \sim \pi_\theta(a \mid s_t).$$

For continuous action spaces, the agent samples an action a_t from the Gaussian policy:

$$a_t \sim \mathcal{N}(\mu_\theta(s_t), \sigma_\theta^2(s_t)).$$

To ensure actions remain within valid bounds, a squashing function like tanh may be applied:

$$a_t = \tanh(\tilde{a}_t),$$

where $\tilde{a}_t$ is the raw action sampled from the Gaussian distribution. The squashed action a_t is then scaled to the desired action range.

Log Probability Adjustment for Squashing Functions:

When using continuous action spaces, it’s often necessary to bound the actions to a finite interval (e.g., $[-1, 1]$). This is typically achieved using a squashing

function like $\tanh$. However, applying such a transformation requires adjusting the log probabilities to maintain a proper probability distribution.

The log probability must be adjusted using the change of variables formula:

$$\log \pi_{\theta}(a_t | s_t) = \log \pi_{\theta}(\tilde{a}_t | s_t) - \sum_i \log \left| \frac{d \tanh(\tilde{a}_{t,i})}{dx} \right|,$$

where $\tilde{a}_t$ is the unbounded action before applying $\tanh$, $a_t = \tanh(\tilde{a}_t)$ is the bounded action, and i indexes the action dimensions. The second term is the log of the absolute value of the determinant of the Jacobian of the transformation.

The derivative of $\tanh$ used in the adjustment is:

$$\frac{d \tanh(x)}{dx} = 1 - \tanh^2(x).$$

This adjustment ensures that we properly account for the “compression” of the probability density that occurs when mapping from an unbounded to a bounded space. Without this correction, the policy gradients would be incorrect, potentially leading to suboptimal learning.

Stability:

To ensure stable learning, PPO constrains policy updates using the clipping mechanism in the surrogate objective. The agent monitors stability by checking if the change in policy parameters falls below a specified tolerance:

$$\|\theta_{\text{new}} - \theta_{\text{old}}\|_2 < \text{tol}$$

where tol is a predefined tolerance threshold.

2.4 Metrics Implemented

We have implemented 5 metrics which can be used to study how algorithms performed for each agent over multiple learning trajectories. We include an example with each metric showing what the output looks like. All these examples are for Q-learning vs Q-learning over 10 learning trajectories.

The environment was set up using the default model used in Calvano [3], where the following parameters were used: `tstable` = 100000, `tmax` = 1000000. Both the Agents were initialized with the action space

$$A = [1, 1.125, 1.25, 1.375, 1.51.625, 1.75, 1.875, 2.] \quad (4)$$

and $\beta = 10^{-4}$.

2.4.1 Average prices and profits.

This function uses the history of all the trajectories of the trained agents and calculates the average price and its standard deviation charged by each agent. It also does the same with the profit gained by each agent. It is important to note that this function does not average over the entire histories of the learning trajectories, but instead only averages over the last stable period defined in the

environment.

This function can display the average Δ value, as defined in Calvano 2020 [3], for each agent.

$$\Delta \equiv \frac{\bar{\pi} - \pi^N}{\pi^M - \pi^N},$$

where $\bar{\pi}$ is the average per-firm profit upon convergence, π^N is the profit in the Bertrand-Nash static equilibrium, and π^M is the profit under full collusion (monopoly). Thus, $\Delta = 0$ corresponds to the competitive outcome and $\Delta = 1$ to the perfectly collusive outcome.

All values are averaged over the last `tstable` steps and all trajectories. Table 1 gives an example of how the output for this metric would look for 2 Q-learning agents.

Average Pricing Metric	Q-learning	Q-learning
<i>Price Statistics</i>		
Average	1.6655	1.6635
Standard Deviation	0.0579	0.0589
<i>Profit Statistics</i>		
Average	0.2917	0.2930
Standard Deviation	0.0169	0.0163
<i>Delta Statistics</i>		
Average	0.6000	0.6120
Standard Deviation	0.1479	0.1420

Table 1: Average prices, profits, and delta per agent using the Average Pricing Metric for Q-learning vs Q-learning.

2.4.2 State Heat Map

This function only works when both agents use discrete action spaces. It uses the history of the learning trajectories and displays the frequency of each chosen price pairs in a heat map over their stable period. This shows us how these algorithms act during their stable period. A single concentrated cell implies that the algorithms stabilize to a single price pair, a more varied distribution could imply cycles, or a large amount of states being visited could also imply the algorithms need more time to stabilize. Figure 1 is an example of the heat map metric.

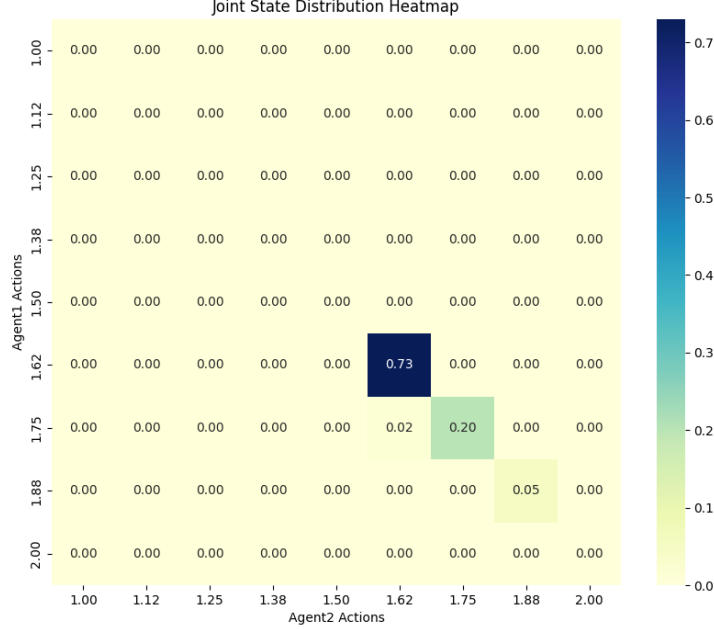


Figure 1: Example of the heat map metric for Q-learning vs Q-learning

2.4.3 Make adjacency

This function takes the Q-values of any two agents which learn using Q-values and a history dependent state space, such as Q-learning or Decentralized Q-learning, and then makes an adjacency matrix for all possible price pairs for a single learned trajectory. This is an interesting measure to get a more visual representation of how the agents interact with each other.

In the adjacency matrix, each column and row represent a joint action space of the two agents. All sets of prices $p_1, p_2 \in A_1 \times A_2$ are indexed from 0 to $|A_1| * |A_2| - 1$. Let $i \in [0, |A_1| * |A_2| - 1]$ be the index associated with a price pair $p_1, p_2 \in A_1 \times A_2$. Then for all $j \in [0, |A_1| * |A_2| - 1]$, the cell $(i, j) = 1$ if the prices $p_3, p_4 \in A_1 \times A_2$ associated with j are the greedy outputs of the learned policies of the two agents given p_1, p_2 , else cell $(i, j) = 0$.

This adjacency matrix can also be displayed as a network graph where the nodes are the price pairs, and the edges point to the next price pair resulting from the previous states. Figure 2 is an example.

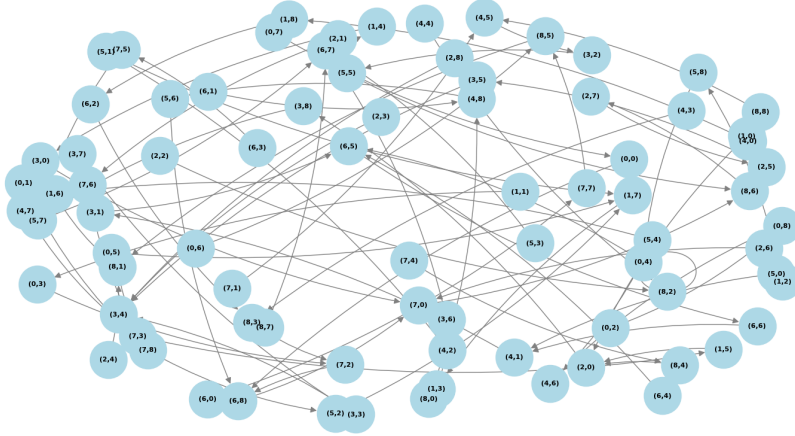


Figure 2: Example of the Make adjacency metric plotted for Q-learning vs Q-learning

2.4.4 Reward-Punishment Scheme

In Joseph E. Harrington Jr.'s book *The Theory of Collusion and Competition Policy* [18], he argues that reward-punishment schemes are necessary to sustain collusive prices. Therefore, this function uses the adjacency matrix of the Q-learning agents, made using the previous metric, to check if they follow a reward punishment scheme. Using the two papers written by Meylahn [19, 20], an adjacency matrix shows a reward-punishment scheme if (1) The largest component of the graph has the Nash price pair included, (2) and another price pair, higher than the Nash price pair must be the limiting node or in the limiting cycle of the graph. Note that this definition is a looser modification of the definitions used in the papers.

The strategy graphs (from the make adjacency metric) employed here constitute functional relations. When each state (action pair) is represented as a node, and directed edges are drawn from each state to the state reached by both players executing their greedy action, each state has exactly one outgoing edge (out-degree of one). This structural property implies that the strategy graphs decompose into components, where each component comprises trees converging towards a single cycle or self-loop. To characterize reward and punishment schemes that maintain supra-competitive prices, the critical factors are identifying which component of the strategy graph will dominate the players' time distribution when exploration rates are considered, and whether that dominant component leads to supra-competitive prices.

This function takes the Q-values of the history Q-learning agents over all trajectories, and then graphs what fraction of the trajectories show this scheme over time.

Reward-Punishment schemes are often used in the literature, however, there

are doubts regarding whether they are required for collusion. A paper by Lambin [21] suggests that supracompetitive prices may not be a result of the reward-punishment schemes but instead both phenomena may originate from simultaneous experimentation and learning inertia inherent in reinforcement learning, without a causal link between them.

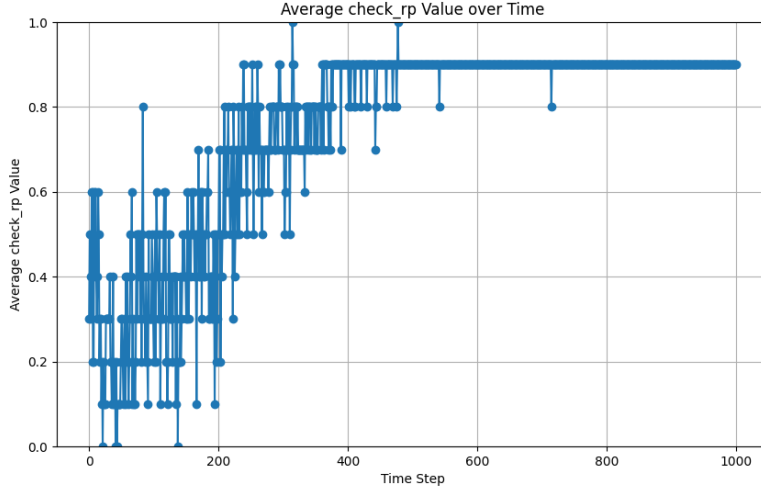


Figure 3: Example of the Reward-Punishment for Q-learning vs Q-learning. Where the y-axis shows the ratio of trajectories which show this reward-punishment scheme at different learning time steps

2.4.5 Regret metric

One of the indications of collusion mentioned in the paper by den Boer and Meylahn [10] is collusive algorithms having low regret against alternative algorithms. This is an important property since a firm is only likely to choose a collusive algorithm if it performs well against a reasonable alternative.

Regret is defined as the difference between an algorithm's payoff and the payoff under the optimal fixed action in hindsight. Write

$$r_i(a_i, a_{-i}, \nu) := \mathbb{E}_\nu[f(a_i, Y_{i, a_i, a_{-i}})]$$

for the expected reward of player i as function of (a_i, a_{-i}) and ν (the environment). Then, for all strategies $(\pi_i, \pi_{-i}) \in \Pi_i \times \Pi_{-i}$, $\nu \in \mathcal{V}$ and $i = \pm 1$, we define regret by

$$\text{Regret}_i(\pi_i, \pi_{-i}, \nu) := \sup_{a \in A_i} \mathbb{E}_\nu^{\pi_i, \pi_{-i}}[\Phi((r_i(a, A_{-i}(t), \nu))_{t \in \mathbb{N}})] - \phi(\pi_i, \pi_{-i}, \nu).$$

where $\phi(\pi_i, \pi_{-i}, \nu)$ is the cumulative reward in environment ν for agent i playing policy π_i against agent $-i$ playing policy π_{-i} .

The paper also uses another definition of regret, if rewards are strictly positive with probability one, regret could be defined in terms of the *ratio* between these two quantities:

$$\text{Regret}'_i(\pi_i, \pi_{-i}, \nu) := 1 - \frac{\phi_i(\pi_i, \pi_{-i}, \nu)}{\sup_{a \in A_i} \mathbb{E}_{\nu}^{\pi_i, \pi_{-i}} [\Phi((r_i(a, A_{-i}(t), \nu))_{t \in \mathbb{N}})]}.$$

both this definitions of regret were implemented in the gym, labeled as ‘subtract’ and ‘ratio’ respectively.

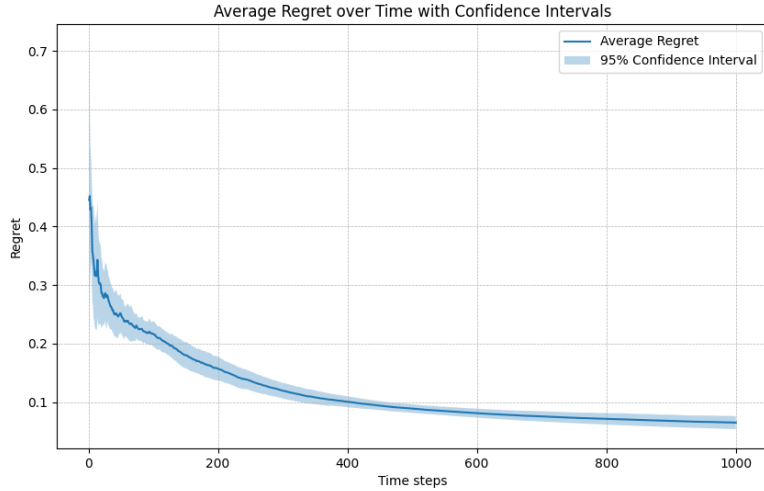


Figure 4: Example of the regret ratio graph for Q-learning vs Q-learning.

3 Results

Two experiments were conducted to test different aspects of this gym. The first one was to run Q-learning against SGD with a continuous state space. The main reason this experiment was run was to showcase how this new adaptation of state space management (see Section 2.3.1) can be used to run discrete space algorithms such as Q-learning against unknown prices. The second experiment was to test for collusive properties of the Q-learning algorithm using the gym’s features. This was the main motivation for building the gym. Although the gym cannot test for all 6 properties of collusive algorithms stated in den Boer and Meylahn’s paper [10], this experiment showcases how this gym can be used to test for 3 out of the 6 properties.

3.1 Experimental Setup

For all the experiments, the algorithms were run for 48 learning trajectories. Each trajectory was run for a maximum of one million steps, but the learning was stopped if both algorithms were stable for one hundred thousand steps. The environment set up was the same as the one used by Calvano et al. [3], with the Nash price being 1.473 and the Monopoly price being 1.925 for both agents. For all the algorithms with discrete state spaces used in this experiment (Q-learning, and EXP3), the action space was always

$$A = [1, 1.125, 1.25, 1.375, 1.51.625, 1.75, 1.875, 2] \quad (5)$$

. For the decaying exploration rate of Q-learning, a β value of 10^{-4} was used. Note that from prior testing, 10^{-4} is not small enough for full exploration, and a β value of at most 10^{-5} should be used. However, for this β value, not a single trajectory stabilized within a million iterations, therefore a higher value was chosen for the demo. Also, for storage optimization, the action history of each trajectory was recorded at every 1000 steps instead of the full history.

3.2 Experiment 1: Q-learning vs SGD

This experiment was conducted by running Q-learning with 3 different state space settings, 48 times each, and then observing the average profits using the average price metric. The SGD algorithm had a continuous action space of $A_2 = [1, 2]$, the same bounds as the discrete action space. The 3 settings used were, `lump_tol` = 0.01, `lump_tol` = 0.06, and `lump_tol` = ∞ (approximate unknown price to the closest values in the state space). Here Agent 1 is the Q-learning agent and Agent 2 is the SGD agent.

Table 2: Q-learning vs SGD: Average values and standard deviations in parenthesis under different `lump_tol` Settings

<code>lump_tol</code>	Agent 1 Price	Agent 2 Price	Agent 1 Profit	Agent 2 Profit	Total Time (s)
∞	1.5533 (0.1022)	1.3027 (0.2366)	0.1546 (0.0723)	0.1774 (0.1379)	7585.37
0.06	1.5303 (0.1015)	1.2800 (0.2500)	0.1511 (0.0786)	0.1570 (0.1392)	7721.64
0.01	1.5312 (0.1038)	1.2689 (0.2493)	0.1467 (0.0779)	0.1526 (0.1408)	7128.37

The results show no real differences in the prices or the profits for either agents. The main reason for this could be that the discrete action space chosen for the Q-learning agents is already very finely discretized around the Nash and the Monopoly prices such that adding more states does not make a big difference. The difference in the time required for learning and stabilization is also lower than expected. This lack of differences may be due to the relatively fine discretization of the action space employed in all the experiments. This is a good demo for how this new state space management system can be used to run a discrete learning algorithm against a continuous learning algorithm, further experiments can be conducted to test if the lumping truly shows any significant changes to the result and whether the time and space cost is worth it.

3.3 Experiment 2: Test for properties of collusion

The first requirement for collusion was supracompetitive payoffs - if both players use a collusive algorithm, both players' payoffs are supracompetitive. This was tested by running Q-learning against itself and observing the average payoff metric. The result is shown below.

Average Pricing Metric	Q-learning	Q-learning
<i>Price Statistics</i>		
Average	1.7177	1.6786
Standard Deviation	0.1311	0.1324
<i>Profit Statistics</i>		
Average	0.2729	0.3045
Standard Deviation	0.0582	0.0654
<i>Delta Statistics</i>		
Average	0.4367	0.7217
Standard Deviation	0.2488	0.2952

Table 3: Average prices, profits, and delta per agent using the Average Pricing Metric for Q-learning vs Q-learning

Here both players do indeed play supracompetitive prices on average, although not exactly the monopoly prices.

The second requirement was a good performance against alternative algorithms: collusive algorithms have low regret against alternative algorithms. Here the alternative algorithm chosen was EXP3 with the same action space as Q-learning and the regret was plotted.

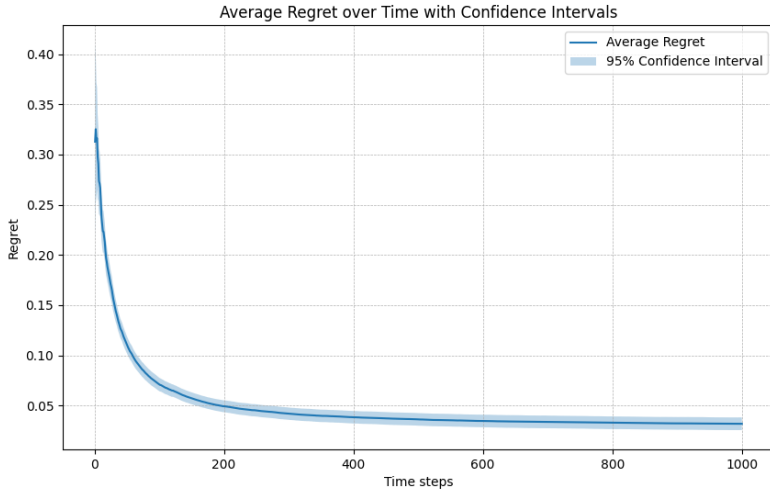


Figure 5: Regret of Q-learning against EXP3.

Figure 5 shows that as the time steps increase, the regret decreases over time, meaning that as the algorithm learns, its payoffs are closer to the best static strategy in hindsight. However, it does not massively decrease to zero, which would be the desirable output.

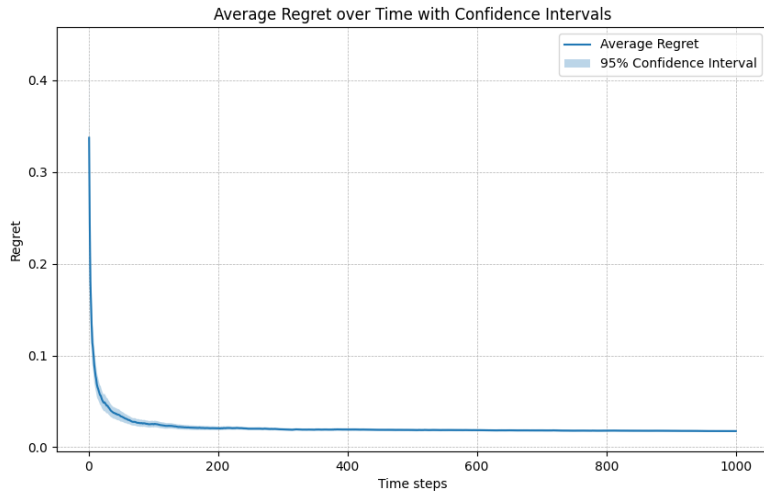


Figure 6: Regret of Exp3 against Q-learning.

Since Exp3 is a no regret algorithm, we expect the plot to go to zero, however, here it seems to converge to some constant. One reason for this could be the way this graph is plotted. Since not all trajectories are of the same length, the last regret value of each trajectory is extended till the end of the largest trajectory before averaging. Therefore if 40/50 algorithms stabilized by 800 time steps at around 0.05, then this value will be copied till the end.

The third requirement for collusion is Deterrence: collusive algorithms generate higher payoff than alternative algorithms in case the opponent uses a collusive algorithm. Here, the average payoffs of Q-learning vs EXP3 was calculated, where Agent 1 is Q-learning and Agent 2 is EXP3.

Average Pricing Metric	Q-learning	EXP3
<i>Price Statistics</i>		
Average	1.6566	1.6512
Standard Deviation	0.1331	0.1379
<i>Profit Statistics</i>		
Average	0.2790	0.2797
Standard Deviation	0.0588	0.0504
<i>Delta Statistics</i>		
Average	0.4900	0.4961
Standard Deviation	0.2458	0.3885

Table 4: Average prices, profits, and delta per agent using the Average Pricing Metric for Q-learning vs EXP3.

Here the profits for EXP3 against Q-learning is 0.2797, which is in between the profits for the two Q-learning agents run against each other 0.2729 and 0.3045. This shows that Q-learning fails to meet the deterrence requirement for collusion since Q-learning vs Q-learning does not always show higher profits than EXP3 vs Q-learning.

4 Conclusion

This report presents a flexible reinforcement learning gym environment designed to study algorithmic pricing and collusion. The gym’s architecture, consisting of customizable Environment, Agent, and Metrics components, enables users to systematically investigate how different pricing algorithms interact in various market settings.

The key features of this gym include the ability to play different algorithms against one another, a novel state space management system such that algorithms with discrete state spaces can interact with algorithms of continuous action spaces or a space not equal to their own. This design ensures compatibility between different types of learning algorithms while maintaining realistic market conditions where firms may not have complete information about their

competitors. It also includes various metrics such as the regret metric and the reward-punishment scheme metric.

The experiments demonstrated two primary capabilities of the gym: First, the successful interaction between discrete and continuous space algorithms through Q-learning versus Stochastic Gradient Descent. Second, the gym’s ability to test for three key properties of collusion based on den Boer and Meylahn’s definition: supracompetitive payoffs, low regret against alternative algorithms, and deterrence [10].

The gym does face some technical limitations that need to be addressed. The primary constraint is simulation speed, which becomes particularly apparent when running multiple trajectories or longer training periods. A potential solution would be parallel execution of trajectories, however, this presents challenges with the current architecture. Similarly, while saving complete trajectory data to files would enable more thorough analysis, the current design makes this difficult to implement alongside the simulation process.

Future work should focus on conducting longer training periods with optimized parameters to draw more definitive conclusions about collusive behavior. Additional research directions include implementing more learning algorithms, developing new metrics for detecting collusion, and investigating the impact of different market structures on algorithmic behavior.

References

- [1] Ahmed Rashied, Siham Obaid, Omar Baban, and Aseel Muhsin. The effects of market structure and competition policy on consumer welfare and economic efficiency. *Journal of Ecohumanism*, 3:949–962, 09 2024. doi: 10.62754/joe.v3i5.3948.
- [2] Ma. Joy V. Abrenica. Balancing consumer welfare and public interest in competition law. *Asian Journal of WTO & International Health Law and Policy*, 13(2):443–462, September 2018. URL <https://ssrn.com/abstract=3256737>.
- [3] Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10):3267–97, October 2020. doi: 10.1257/aer.20190623. URL <https://www.aeaweb.org/articles?id=10.1257/aer.20190623>.
- [4] A Autier, A Ittoo, and P Van Cleynenbreugel. Ai algorithms, price discrimination and collusion: a technological, economic and legal perspective. *European Journal of Law and Economics*, 50:405–435, 2020. URL <https://link.springer.com/article/10.1007/s10657-020-09662-6#Bib1>.
- [5] Tim Loots and Arnoud V. den Boer. Data-driven collusion and competition in a pricing duopoly with multinomial logit demand. *Production and Operations Management*, 32(4):1169–1186, 2023. doi: 10.1111/poms.13919.
- [6] Wolfram Barfuss and Janusz M. Meylahn. Intrinsic fluctuations of reinforcement learning promote cooperation. *Scientific Reports*, 13(1309), 2023. doi: 10.1038/s41598-023-27672-7.
- [7] Ibrahim Abada and Xavier Lambin. Artificial intelligence: Can seemingly collusive outcomes be avoided? 2020. URL <https://ssrn.com/abstract=3559308>. SSRN Working Paper.
- [8] Timo Klein. Autonomous algorithmic collusion: Q-learning under sequential pricing. *RAND Journal of Economics*, 2019. doi: 10.2139/ssrn.3195812. URL <https://ssrn.com/abstract=3195812>. Forthcoming.
- [9] Ibrahim Abada, Joseph E. Harrington Jr, Xavier Lambin, and Janusz M Meylahn. Algorithmic collusion: Where are we and where should we be going? 2024. doi: 10.2139/ssrn.4891033. URL <https://ssrn.com/abstract=4891033>. SSRN Working Paper.
- [10] Arnoud V. den Boer and Janusz M. Meylahn. A (Mathematical) definition of algorithmic collusion. November 2024. doi: 10.2139/ssrn.5012923. URL <https://ssrn.com/abstract=5012923>.

- [11] L. Waltman and U. Kaymak. Q-learning agents in a Cournot oligopoly model. *Journal of Economic Dynamics and Control*, 32(10):3275–3293, 2008.
- [12] C.J.C.H. Watkins and P. Dayan. Q-learning. *Mach Learn*, 8:279–292, 1992. doi: 10.1007/BF00992698.
- [13] Muhammed O. Sayin, Kaiqing Zhang, David S. Leslie, Tamer Basar, and Asuman E. Ozdaglar. Decentralized q-learning in zero-sum markov games. 2021. URL <https://openreview.net/forum?id=nhkbYh30T1>.
- [14] Gurdal Arslan and Serdar Yuksel. Decentralized Q-learning for stochastic teams and games. *IEEE Transactions on Automatic Control*, 62(4), April 2017.
- [15] Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. The nonstochastic multiarmed bandit problem. *SIAM Journal on Computing*, 32(1):48–77, 2002. doi: 10.1137/S0097539701398375.
- [16] Rashmi Ranjan Bhuyan, Adel Javanmard, Sungchul Kim, Gourab Mukherjee, Ryan A. Rossi, Tong Yu, and Handong Zhao. Structured dynamic pricing: Optimal regret in a global shrinkage model. *Journal of Machine Learning Research*, 25(224):1–46, 2024. URL <http://jmlr.org/papers/v25/23-1365.html>.
- [17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *ArXiv*, abs/1707.06347, 2017. URL <https://api.semanticscholar.org/CorpusID:28695052>.
- [18] Joseph E. Harrington Jr. *The Theory of Collusion and Competition Policy*. The MIT Press, November 2017. ISBN 9780262343008.
- [19] Janusz M Meylahn. Does an intermediate price facilitate algorithmic collusion? August 2024. URL <https://ssrn.com/abstract=4594415>. Available at SSRN.
- [20] Janusz M Meylahn. Quantifying the likelihood of learning collusive strategy equilibria. August 2024. URL <https://ssrn.com/abstract=4589989>. Available at SSRN.
- [21] Xavier Lambin. Less than meets the eye: simultaneous experiments as a source of algorithmic seeming collusion. January 2024. URL <https://ssrn.com/abstract=4498926>. Available at SSRN.
- [22] Vedant Mainkar. RL_pricing_gym: A gym to test rl algorithms in pricing environments. https://github.com/E-Schrodinger/RL_Pricing_Gym, 2024.

A Running Your Own Experiments

This appendix explains how to use the gym to run pricing experiments. Find the code here: `RL_Pricing_Gym` [22].

A.1 Environment Selection

The environment defines the market setting. The IRP (Logit demand model) environment is currently available:

```
game = IRP(tmax=1000000, tstable=100000)
```

Parameters: `tmax`: Total simulation length `tstable`: Stabilization period

A.2 Agent Selection

The gym offers several learning algorithms. Users can choose from Q-Learning (standard Q-learning algorithm), Dec-Q (decentralized Q-learning), Exp3 (exponential weights algorithm), SGD (Stochastic Gradient Descent), and PPO (proximal policy approximation).

Agent parameters:

```
Agent = Q_Learning(game,
                    beta,          # learning rate
                    Qinit,        # Q-table initialization
                    a1_prices)    # action space
```

A.3 Metrics and Analysis

The gym provides several metrics for analyzing results:

Average Price and Profit:

```
stats = average_price_and_profit(game, a1_list,
                                a2_list, ts)
```

Returns mean and standard deviation of prices and profits.

Regret Analysis:

```
regret = find_regret(game, Agent1, Agent2,
                    a1_list, a2_list,
                    use_loglog, regret_type)
```

Calculates cumulative or average regret.

State Heatmap:

```
state_heatmap(game, Agent1, Agent2,
              a1_list, a2_list, time_step)
```

Visualizes state space exploration.

Price Trajectory:

```
plot_rp(a1_list, a2_list, time_step)
```

Plots price evolution over time.

A.4 Example Configuration

Basic setup for comparing two agents:

```
# Environment
game = IRP(tmax=1000000, tstable=100000)

# Action space
d_action_space = [1.0, 1.125, 1.25, 1.375, 1.5,
                  1.625, 1.75, 1.875, 2]

# Agents
Agent1 = Q_Learning(game, beta=0.0001,
                    Qinit='uniform',
                    a1_prices=d_action_space)
Agent2 = Exp3(game, beta=0.0001,
              Qinit='uniform',
              a1_prices=d_action_space)

# Simulation
SM = Simulations(game, Agent1, Agent2,
                 iterations=48, ts=1000)
a1_list, a2_list = SM.get_values()

# Analysis
stats = average_price_and_profit(game, a1_list,
                                a2_list, ts=1000)
regret = find_regret(game, Agent1, Agent2,
                    a1_list, a2_list)
```